Project Summary
InvisiBits - Steganography Tool

Laura Levistö
5/25

InvisiBits is a web-based steganography tool that allows users to encode and decode hidden messages in images (PNG, JPG) using the Least Significant Bit (LSB) technique. The tool provides optional password-based encryption for added security, which is handled on the client side using JavaScript (CryptoJS). The backend never sees the password or unencrypted message. The interface is user-friendly, with robust error handling and cybersecurity measures.

Logic and the Flow

Encoding

Front

- Users upload an image and input a message to encode.
- Optionally, users can provide a password for encrypting the message.
- If a password is provided, the message is encrypted in the browser using CryptoJS (AES).
- The (optionally encrypted) message is sent to the backend via an HTTP POST request to the `/encode` route.

Back

- The `/encode` route in app.py validates the input file and message.
- The backend embeds the message as-is (already encrypted if a password was used) using the LSB technique via `encode_message_in_image` in `encoder.py`.
- The modified file is saved with `_steg` appended to the filename and made available for download.

Output

- The encoded file is saved automatically.

Decoding

Front

- Users upload a steganographic image and optionally provide a password for decryption.
- The frontend sends the file to the backend via an HTTP POST request to the `/decode` route.
- If a password was used, the frontend decrypts the message using CryptoJS (AES).

Back

- The `/decode` route in app.py validates the input file.
- The backend extracts the message as-is using `decode_message_from_image` in `decoder.py` and returns it to the frontend.

Output

- The decoded message is displayed in a popup and can be saved as a `.txt` file.

Check

Front

- Users upload an image to check for hidden messages.
- The frontend sends the file to the `/check` route.

Back

- The `/check` route analyzes the image to determine if it contains a hidden message by checking the LSBs of the first 32 pixels for a valid message length.

Output

- The result ("Image has a hidden message" or "No hidden messages found") is displayed in the interface.

Why Python is Used

- - Python's simplicity and readability make it ideal for rapid development.
- - It has robust libraries like `Flask` for web development and `Pillow` for image processing.
- - Python is platform-independent, ensuring the tool works on various operating systems.

How the Interface is Embedded with Python

- - The `index.html` file is served by Flask using the `render_template` function.
- - Static files (CSS, JavaScript, images) are served from the static directory.
- - User actions on the interface trigger AJAX requests to Flask routes (`/encode` and `/decode`), which process the requests and return responses.

Why Each Library is Used

- **Flask:** Provides a lightweight framework for building the web application, handling routing, request processing, and serving static files.
- **Pillow:** Used for image processing, including reading, modifying, and saving images. Essential for implementing the LSB steganography technique.

File and Function Relationships

Front

- **`index.html`**: Provides the structure of the web interface, including input fields, buttons, and sections for encoding, decoding, and checking.
- **`main.js`**: Handles user interactions, validates inputs, and sends requests to the backend, client-side encryption/decryption (CryptoJS). It also updates the UI based on backend responses.
- **`style.css`**: Styles the interface, ensuring it is visually appealing and user-friendly.

Back

- **`app.py`**: The main Flask application that handles requests from the frontend and routes them to the appropriate encoding, decoding, or checking functions.
- **`encoder.py`**: Contains the `encode_message_in_image` function for embedding messages into images.
- **`decoder.py`**: Contains the `decode_message_from_image` function for extracting messages from images.

Cybersecurity Aspects

Password Protection:

- Optional password-based encryption is performed in the browser. The password and unencrypted message are never sent to the server.

XSS Protection

- User inputs are sanitized using DOMPurify.

Error Handling

- Robust error handling provides meaningful feedback.

File Validation:

- Only supported file formats (PNG, JPG) are processed.
- File size is limited to 5MB.

Concepts Explained

Least Significant Bit (LSB) Steganography, Encryption, and Digital Forensics

The InvisiBits tool uses the Least Significant Bit (LSB) technique to embed secret messages within images. This method works by modifying the smallest bit of each pixel's color value to encode binary data from the message. Because the change is so minimal, it is imperceptible to the human eye—meaning the image looks unchanged, but contains hidden information. For example, a pixel value like `(10101010, 11001100, 11110000)` might become `(10101011, 11001101, 11110001)` after embedding data in the LSBs.

Steganography (hiding the existence of a message) and encryption (making a message unreadable without a key) serve different purposes. Steganography conceals the presence of communication, while encryption protects its content. InvisiBits allows you to combine both: a message can be encrypted in the browser before being hidden in an image, so even if the hidden data is discovered, it cannot be read without the password.

The encoding and decoding algorithms used by InvisiBits are efficient, operating in linear time (`O(n)`, where `n` is the number of pixels in the image). Each pixel is processed only once, ensuring the tool can handle large images quickly and with minimal computational overhead.

This technique is highly relevant to digital forensics. Investigators may need to detect or extract hidden messages in digital media during cybercrime investigations or to prevent data exfiltration. Understanding LSB steganography helps forensic experts uncover concealed data that might otherwise go unnoticed.

Password

- The tool applies AES encryption for password-protected messages
- It fails to decode encrypted messages without the correct password..
- The encryption algorithm uses a 32-byte key and CBC mode for security.

User Interface

- Intuitive Design
- - Tabs for encoding, decoding, and checking make navigation simple.
- - Clear labels and placeholders guide users through the process.

- Error Handling
- - Provides clear feedback for invalid inputs (e.g., empty fields, unsupported file formats).
- - Displays error messages for failed operations (e.g., incorrect password).

- Accessibility
- - Character counters for input fields help users stay within limits.
- - Password toggle buttons improve usability.

- Encoded images are visually identical to the original.
- The tool successfully extracts hidden messages from PNG and JPG images.

Encryption Logic

- - If a password is provided, the message is encrypted in the browser using CryptoJS (AES) before being sent to the server.
- - The backend embeds/extracts the message as-is, without knowing the password or plaintext.
- - The password is never sent to the server.
- - All encryption and decryption is performed client-side in JavaScript. The server only handles embedding and extracting the (optionally encrypted) message.

Encode Logic

Image size checks:
- Image must have at least 32 pixels (to store message length).
  Error: `"Image is too small to check for hidden messages."`
- Image must not exceed 9 million pixels (3000×3000) or 5MB.
  Error: `"Image dimensions are too large. Please use a smaller image."` or `"File size exceeds the maximum limit of 5MB."`

Message size checks:
- Message (after encryption if password is used) must fit:
  `total_required = 32 + len(message_binary)`
  Error: `"Message is too long for this image."`

Password:
- If provided, message is encrypted client-side (in JS), making it much longer.
- Encrypted messages can easily exceed the capacity of small/medium images.

Results by Image Size:

Image Size
Can Encode? (No Password)
Can Encode? (With Password)
Typical Result/Reason
Error Message(s)

10×10 (100 px)
Only a few chars
Usually fails
Too small for most messages
"Message is too long for this image."

16×16 (256 px)
Short message
Often fails
Encrypted message too big
"Message is too long for this image."

32×32 (1024 px)
Short message
Maybe short message
Capacity: (1024-32)/8 ≈ 124 chars
"Message is too long for this image."

256×256
Up to ~8,190 chars
Fewer if encrypted
Normal use, but encryption reduces cap
"Message is too long for this image."

512×512
Up to ~32,510 chars
Fewer if encrypted
Most messages fit
"Message is too long for this image."

800×600
Up to ~59,968 chars
Fewer if encrypted
Large messages fit
"Message is too long for this image."

1280×720
Up to ~115,168 chars
Fewer if encrypted
Very large messages fit
"Message is too long for this image."

1920×1080
Up to ~258,048 chars
Fewer if encrypted
Very large messages fit
"Message is too long for this image."

3000×3000
Blocked (>5MB or pixels)
Blocked
Too big, rejected by backend/frontend
"Image dimensions are too large..." or "File size..."

Decode Logic

Image size checks:
- Must have at least 32 pixels.
  Error: `"Image is too small to contain a hidden message."`

Reads message length from first 32 pixels:
- If length is 0, negative, or > (pixels-32), error is raised.
  Error: `"Image may not contain a valid hidden message."` or `"Corrupted or invalid hidden message length."`

Extracts message bits:
- If password was used, the extracted message is encrypted and must be decrypted client-side.

Out of range" case:
- If the message length is garbage (from a non-encoded image), the code may try to read pixels that don't exist, causing an **IndexError** or `"Corrupted or invalid hidden message length."`

Password:
- Not required for extraction, but needed to make sense of encrypted messages.

Results by Image Size:

Image Size
Decoding Result (No Password)
Decoding Result (With Password)
Typical Issue/Behavior
Error Message(s)

10×10, 16×16
Fails or gibberish
Fails or gibberish
Not enough pixels for message
"Image is too small to contain a hidden message."

32×32
Works for short messages
Works for short messages
Longer encrypted messages may fail
"Corrupted or invalid hidden message length."

256×256+
Works if message fits
Works if message fits
If password used, must decrypt
"Corrupted or invalid hidden message length."

Too big
Not allowed
Not allowed
Blocked by backend/frontend
"Image dimensions are too large..." or "File size..."

Check Logic

Image size checks:
- Must have at least 32 pixels, not over 9 million pixels or 5MB.
  Error: `"Image is too small to check for hidden messages."` or `"Image dimensions are too large. Please use a smaller image."`

Reads message length from first 32 pixels:
- If length is 0, negative, or > (pixels-32), returns "no hidden message" or error.
  Error: `"No hidden messages found."`

Out of range" case:
- If the message length is garbage (from a non-encoded or random image), the code returns `"No hidden messages found."`

Does not care about password:
- Only checks for presence of a message, not its content.

Results by Image Size:

Image Size
Check Result
Typical Issue/Behavior
Error Message(s)

10×10, 16×16
"Too small" error
Not enough pixels for check
"Image is too small to check for hidden messages."

32×32
"No hidden message" or OK
Only detects if message present
"No hidden messages found."

256×256+
"No hidden message" or OK
Works as expected
"No hidden messages found."

Too big
"Image too large" or "too big" error
Blocked by backend/frontend
"Image dimensions are too large..." or "File size..."

Password Impact

Encoding:
- Password encrypts the message, making it much longer.
- Small images that could fit a short plain message may not fit the encrypted version.

Decoding:
- Extracted message is encrypted if password was used.
- User must provide the correct password to decrypt and read the message.

Check:
- Password is irrelevant; only presence of a message is detected.

Practical Examples

10×10 image:
- Can’t encode more than a few characters, fails with password.
- Check returns `"Image is too small to check for hidden messages."`
- Decode returns `"Image is too small to contain a hidden message."`

256×256 image:
- Can encode a normal message (e.g., 200 chars) without password.
- With password, may fail if encrypted message is too long.
- Check works, decode works if password is correct.

1920×1080 image:
- Can encode very large messages, even with password.
- Check and decode work as expected.

3000×3000 image:
- Blocked by frontend/backend due to size limits.
- Error: `"Image dimensions are too large. Please use a smaller image."` or `"File size exceeds the maximum limit of 5MB."`

Non-encoded or random image:
- Check may return `"No hidden messages found."`
- Decode may return `"Image may not contain a valid hidden message."` or `"Corrupted or invalid hidden message length."`

XSS Protection Testing

To verify that user input is properly sanitized and XSS attacks are prevented:

1. Enter the following payloads in the message or password fields:

   - `<script>alert('XSS');</script>`
   - `<img src="x" onerror="alert('XSS')">`
   - `<svg onload="alert('XSS')">`

2. Encode and decode as usual.

Expected Result:
The payloads are displayed as plain text in the output (e.g., as a password or hidden message), and are **not executed** as code. No alerts or popups should appear, and no images or SVGs should render.

This confirms that DOMPurify is effectively protecting against XSS vulnerabilities.

Special Characters

- Use the web UI or API to encode a message containing:
  - Emojis (😍🚀💡), `😀😃😄😁`
  - Non-ASCII letters (ä, ö, å, é, ñ, ü)
  - Non-Latin scripts (你好, привет, مرحبا), `こんにちは世界`
  - Symbols (`@#$%&*`), `!@#€%&/()=?`

Hello 😍! Tämä on testi: ä, ö, å, 你好, 🚀